

TD : Construction d'un Émulateur de CPU en Rust

Ce TD vous guidera pas à pas dans la construction d'un émulateur de CPU, inspiré du CHIP-8, une architecture de CPU simple des années 1970. Ce projet vous permettra de comprendre les concepts fondamentaux de la programmation système, tels que la mémoire, les registres, les opcodes, la pile et l'exécution d'instructions.

Le livre « Rust in Action » utilise le développement incrémental pour ce projet, ce que nous allons reproduire ici.

1 Objectifs d'Apprentissage

À la fin de ce TD, vous devriez être capable de :

- Modéliser une architecture de CPU simple en Rust en utilisant des `struct` et des tableaux.
- Interpréter et décoder des codes opérations (opcodes) binaires.
- Simuler l'interaction entre un CPU et la mémoire.
- Implémenter des fonctions de base d'un CPU (lecture, exécution, gestion de la pile).
- Comprendre l'importance des opérations bit-à-bit en programmation système.
- Manipuler des pointeurs et gérer la mémoire de bas niveau (bien que simplifiée par Rust).
- Gérer des scénarios d'erreur comme le dépassement d'entier (integer overflow).

2 Configuration du Projet

Pour suivre ce TD, vous pouvez soit partir de zéro, soit utiliser les bases de code fournies par le livre.

1. **Cloner le dépôt du livre** (recommandé pour les débutants) :
Vous trouverez les différentes versions du projet CPU dans les sous-dossiers `ch5-cpu1`, `ch5-cpu2`, `ch5-cpu3`, `ch5-cpu4`.
2. **Créer un nouveau projet Cargo** :
Vous ajouterez le code dans `src/main.rs`.

3 Partie 1 : CPU RIA/1 - L'Additionneur

L'objectif de cette première partie est de construire une CPU minimale capable d'effectuer une seule instruction : l'addition.

3.1 Concepts Clés

- **Registres (registers)** : Petites zones de stockage directement accessibles par le CPU, utilisées pour des opérations rapides. Pour le CHIP-8, chaque registre est une valeur `u8`.
- **Opcode** : Un nombre qui représente une instruction et ses opérandes. Les opcodes CHIP-8 sont des valeurs `u16`.

3.2 Tâches

1. Définir la structure CPU :

Créez une `struct CPU` qui contient un champ `current_operation: u16` pour l'opcode actuel et un tableau `registers: [u8; 2]` pour les deux registres que nous utiliserons.

2. Initialiser la CPU et charger des valeurs :

Dans la fonction `main`, créez une instance mutable de CPU. Chargez une valeur dans chaque registre (par exemple, `registers[0] = 5; registers[1] = 10;`). Chargez l'opcode d'addition `0x8014` dans `current_operation`.

3. Décoder l'opcode :

Les opcodes CHIP-8 sont divisés en « nibbles » (4 bits). L'opcode `0x8014` est interprété comme `c=0x8, x=0x0, y=0x1, d=0x4`.

- `c` (le premier nibble) indique le groupe d'opcodes (ici `0x8` pour les opérations sur les registres).
- `x` (le second nibble) indique le premier registre opérande (ici `registers[0]`).
- `y` (le troisième nibble) indique le second registre opérande (ici `registers[1]`).
- `d` (le quatrième nibble) indique le sous-type d'opération (ici `0x4` pour l'addition).

Utilisez les opérations bit-à-bit (`&` et `>>`) pour extraire ces nibbles de `current_operation`.

4. Implémenter l'opération d'addition :

Créez une méthode `add_xy(&mut self, x: u8, y: u8)` dans un bloc `impl CPU`. Cette méthode doit ajouter la valeur de `registers[y]` à `registers[x]`. Convertissez les index `x` et `y` en `usize` pour accéder au tableau.

5. Exécuter l'opcode :

Créez une méthode `run(&mut self)` dans `impl CPU`. Dans `run`, lisez `self.current_operation`. Utilisez une expression `match` sur les nibbles `((c, x, y, d))` pour identifier l'opcode (`0x8, _, _, 0x4`) et appeler `self.add_xy(x, y)`. Pour les opcodes non encore implémentés, utilisez `todo!(« opcode {:#04x} », opcode)`. Dans `main`, appelez `cpu.run()`.

6. Vérifier le résultat :

Utilisez `assert_eq!(cpu.registers[0], 15);` pour confirmer que l'addition a eu lieu. Affichez le résultat avec `println!`.

4 Partie 2 : CPU RIA/2 - Le Multiplicateur (avec mémoire)

Dans cette partie, nous allons améliorer notre CPU pour qu'elle puisse exécuter une séquence d'instructions stockées en mémoire, et introduire la gestion du dépassement d'entier.

4.1 Concepts Clés

- **Mémoire (memory)** : Un tableau de bytes (`[u8]`) où sont stockées les instructions et les données du programme. Le CHIP-8 a 4096 bytes de RAM (`0x1000` en hexadécimal).
- **Position en mémoire (position_in_memory)** : C'est l'équivalent du « program counter » (compteur de programme), un registre spécial qui contient l'adresse de la prochaine instruction à exécuter.
- **Boucle principale (main loop)** : La boucle continue dans laquelle le CPU exécute son cycle (lire, décoder, exécuter).
- **Dépassement d'entier (integer overflow)** : Lorsque le résultat d'une opération arithmétique dépasse la capacité du type de données.

4.2 Tâches

1. Étendre la structure CPU :

Ajoutez `position_in_memory: usize` et `memory: [u8; 0x1000]` à la struct CPU. Modifiez `registers` pour être un tableau de 16 registres (`[u8; 16]`) comme dans la spécification CHIP-8.

2. Lire les opcodes depuis la mémoire :

Modifiez la méthode `read_opcode(&self) -> u16` pour lire deux bytes consécutifs depuis `self.memory` à partir de `self.position_in_memory`. Combinez ces deux bytes (`u8`) en un `u16` (le premier byte est le plus significatif, `op_byte1 « 8 | op_byte2`).

3. Implémenter une boucle principale complète :

Modifiez la méthode `run(&mut self)` pour utiliser une `loop` infinie. À chaque itération, lisez l'opcode, puis incrémentez `self.position_in_memory` de 2 (car chaque opcode a 2 bytes). Ajoutez une condition d'arrêt à la boucle en vérifiant si l'opcode est `0x0000` (utilisez `return;` pour sortir de la fonction `run`). Modifiez la fonction `main` pour initialiser `memory` (par exemple, `[0; 4096]`) et `position_in_memory` à 0.

4. Gérer le dépassement d'entier :

Dans `add_xy`, utilisez la méthode `overflowing_add` de `u8`. `overflowing_add` retourne une paire (`val`, `overflow_detected`). `val` est le résultat et `overflow_detected` est un `bool` indiquant si un dépassement s'est produit. Le CHIP-8 utilise le registre `0xF` (le 16ème registre) comme « flag » de dépassement. Si un dépassement se produit, mettez `registers[0xF]` à 1, sinon à 0.

5. Charger un programme en mémoire :

Dans `main`, chargez une séquence d'opcodes dans `cpu.memory` pour effectuer plusieurs additions. Par exemple :

- `0x8014` (ajouter `reg[1]` à `reg[0]`)
- `0x8024` (ajouter `reg[2]` à `reg[0]`)
- `0x8034` (ajouter `reg[3]` à `reg[0]`)
- `0x0000` (opcode d'arrêt)

6. Vérifier le résultat :

Initialisez des valeurs dans plusieurs registres (`registers[0]=5`, `registers[1]=10`, `registers[2]=10`, `registers[3]=10`). Après l'appel à `cpu.run()`, vérifiez que `registers[0]` contient la somme attendue ($5 + 10 + 10 + 10 = 35$).

5 Partie 3 : CPU RIA/3 - L'Appelleur (avec pile et fonctions)

Cette section introduit la capacité d'appeler et de retourner des fonctions, un élément crucial de toute CPU moderne. Cela implique la gestion d'une pile d'appels.

5.1 Concepts Clés

- **Pile (stack)** : Une région de mémoire utilisée pour stocker temporairement des données, en particulier les adresses de retour des appels de fonction et les variables locales. Fonctionne en LIFO (Last In, First Out).
- **Pointeur de pile (stack_pointer)** : Un registre qui pointe vers le haut de la pile.
- **Opcodes CALL et RETURN** :
 - **CALL** (par exemple, `0x2nnn`) : Pousse l'adresse de retour sur la pile et saute à une nouvelle adresse `nnn` (début de la fonction).
 - **RETURN** (`0x00EE`) : Récupère l'adresse de retour de la pile et saute à cette adresse pour reprendre l'exécution après l'appel.
- **Les fonctions sont des données** : Les instructions du CPU sont des nombres stockés en mémoire, tout comme les données.

5.2 Tâches

1. Étendre la structure CPU :

Ajoutez `stack: [u16; 16]` (la pile du CHIP-8 a une profondeur maximale de 16 appels) et `stack_pointer: usize` à la struct `CPU`. Initialisez `stack_pointer` à 0 dans `main`.

2. Implémenter la méthode `call(addr : u16)` :

Cette méthode est appelée lorsque le CPU rencontre un opcode `CALL`. Stockez l'adresse de la prochaine instruction après l'opcode `CALL` (qui est `self.position_in_memory`) sur la pile (`self.stack[self.stack_pointer]`). Incrémentez `self.stack_pointer`. Mettez à jour `self.position_in_memory` à `addr` (l'adresse de la fonction appelée). Gérez un éventuel `panic!` (« Stack overflow! ») si `stack_pointer` dépasse la taille de la pile.

3. Implémenter la méthode `ret()` :

Cette méthode est appelée lorsque le CPU rencontre un opcode `RETURN`. Décrémentez `self.stack_pointer`. Récupérez l'adresse de retour de la pile (`self.stack[self.stack_pointer]`). Mettez à jour `self.position_in_memory` à cette adresse de retour. Gérez un éventuel `panic!` (« Stack underflow ») si `stack_pointer` est 0 avant de décrémenter.

4. Définir une fonction en mémoire :

Une « fonction » est simplement une séquence d'opcodes. Dans `main`, chargez une petite séquence d'opcodes dans la mémoire à une adresse spécifique (par exemple, à `0x100`). Exemple de fonction : `0x8014` (add reg[1] to reg[0]), `0x8014` (add reg[1] to reg[0]), `0x00EE` (`RETURN`).

5. Modifier la boucle `run` :

Ajoutez des `match` branches pour les opcodes `CALL` (`0x2`, `_`, `_`, `_`) et `RETURN` (`0x0`, `0`, `0xE`, `0xE`). Appelez les méthodes `self.call(nnn)` et `self.ret()` respectivement. N'oubliez pas d'extraire la valeur `nnn` de l'opcode pour l'appel de fonction (`opcode & 0xFFFF`).

6. Tester l'appel de fonction :

Dans `main`, chargez l'opcode `CALL` pour votre fonction (par exemple, `0x2100` pour appeler la fonction à `0x100`). Chargez plusieurs appels si vous le souhaitez pour tester l'imbrication. Assurez-vous que l'exécution se déroule comme prévu et que le registre `registers[0]` contient le bon résultat. L'exemple du livre calcule $5 + (10 \times 2) + (10 \times 2) = 45$.

6 Partie 4 : CPU RIA/4 - Extensions (Facultatif)

Maintenant que vous avez une CPU fonctionnelle avec appels de fonctions, vous pouvez l'étendre avec d'autres opcodes et fonctionnalités.

6.1 Suggestions d'Extensions

— Implémenter plus d'opcodes CHIP-8 :

- Multiplication, soustraction, opérations bit-à-bit (`AND`, `OR`, `XOR`).
- Sauts conditionnels (`if / else` au niveau du CPU en modifiant `position_in_memory` en fonction de la valeur des registres).
- Opérations sur les octets (`0x6xnn` pour définir un registre `Vx` à `nn`, `0x7xnn` pour ajouter `nn` à `Vx`).
- Opcodes d'affichage (le CHIP-8 est connu pour ses graphiques simples).

— Ajouter des entrées/sorties :

- Simuler l'affichage graphique ou la saisie clavier.
- Vous pouvez consulter le code source `ch5/ch5-cpu4` du livre pour une implémentation plus complète.

— Charger des programmes depuis un fichier :

Au lieu de « hardcoder » les opcodes dans `main`, lisez-les depuis un fichier binaire.

— Améliorer la gestion des erreurs :

Distinguer les différents types de `panic!` (stack overflow vs stack underflow).

- **Utiliser des instructions directes au CPU :**

Pour rendre la CPU moins gourmande en énergie, vous pouvez introduire l'instruction `hlt` (halt) pour que le CPU « dorme » en attendant une interruption. Cela peut être fait avec la crate `x86_64` (`x86_64::instructions::hlt`).

7 Concepts Rust Approfondis Utilisés dans ce Projet

Ce projet est riche en concepts Rust cruciaux pour la programmation système :

- **Structures (struct) et Implémentations (impl) :** Pour modéliser le CPU et ses comportements.
- **Tableaux (arrays) :** Utilisés pour les registres et la mémoire du CPU.
- **Opérations bit-à-bit :** Essentielles pour le décodage et l'encodage des opcodes et des données.
- **Expressions match :** Utilisées pour dispatcher l'exécution des opcodes en fonction de leur valeur.
- **Blocs unsafe :** Nécessaires lorsque Rust ne peut pas garantir la sécurité de la mémoire à la compilation, comme pour l'accès direct à la mémoire (`memory`) ou l'utilisation de `std::mem::transmute` (pour interpréter des bits différemment).
- **usize :** Le type d'entier qui correspond à la taille d'un pointeur sur l'architecture cible, idéal pour l'indexation de la mémoire.
- **overflowing_add :** Une méthode sûre pour effectuer des additions qui pourraient dépasser la capacité du type, retournant un `bool` pour indiquer le dépassement.
- **todo! et panic! :** Macros utiles pour les parties non encore implémentées ou pour signaler des erreurs irrécupérables.
- **Pointeurs de fonction :** Bien que souvent implicites, les fonctions elles-mêmes peuvent être traitées comme des adresses en mémoire.
- **Le « Never Type » (!) :** Indique au compilateur qu'une fonction ne retourne jamais (par exemple, parce qu'elle panique ou entre dans une boucle infinie).
- **#[derive(Debug)] :** Permet l'affichage des structures et enums de manière débogable, très utile pour inspecter l'état du CPU.